

- Quiero una agenda para ver a quién tengo que atender.
 - Como usuario:
 - Quiero ver qué turnos están disponibles.
 - Quiero poder reservar un turno.
 - Quiero ver mis turnos reservados.
 - Quiero poder elegir quién me atienda.
 - Quiero un recordatorio antes de mi turno.
 - Quiero una confirmación cuando reservo el turno.
 - Quiero cambiar o eliminar un turno.
 - Quiero crear una cuenta y loguearme.
 - Quiero tener la opción de pagar el turno.
-

En grupo diseñar un proyecto

Identificación del proyecto

Nombre: ECommerce

Versión: 1.0.0

Stack: Express/Node.js - React (Vite recomendado) - MySQL

Librerías: Mysql2 - Axios - Cors - Express-Session - React-Router-Dom - bcryptjs -

Visión general y objetivos

Propósito: Permitir que los usuarios compren cada una de las partes de un producto por separado, para refaccionar otros productos o completar algo de uno faltante. Esta iniciativa también ayuda al medio ambiente. Por eso también tendremos retiro en el local y no envíos.

Objetivos del MVP:

- Búsqueda con diferentes tipos de filtros: El usuario debe poder elegir la categoría de un producto o buscarlo por nombre.
 - Pasarela de pagos: Los usuarios deben tener alguna manera de poder pagar lo que compran.
 - Carrito: En el carrito se deben poder regular las cantidades y se recomendarán productos relacionados.
 - Roles para los usuarios: Que solo el admin pueda cargar productos.
-

Requerimientos del sistema:

Requerimientos funcionales:

- Login y registro: Llevarlo en la base de datos.
- Un CRUD para los productos (solo el admin).
- Pasarela de pagos.
- Carrito.
- Búsqueda de productos.
- Seguridad (por la pasarela de pagos más que nada).

Requerimientos no funcionales:

- Escalabilidad. La estructura debe estar preparada para agregar más funcionalidades, como envíos por ejemplo.
 - Interfaz: El UI/UX debe ser agradable e intuitivo para todo tipo de usuario.
 - Recomendación de productos relacionados a lo que uno compra/tiene en el carrito.
-

Arquitectura de datos:

Tabla Usuarios

Campo	Tipo	Restricciones	Descripción
Id_User	UUID	PK, Default: UUIDV4	Identificador interno
Nombre	Varchar	Not null	Nombre del usuario
Correo	Varchar	Not null, unique	Identificador para el login
Password	Varchar	Not null (va a estar encriptada)	"Llave" para entrar a la cuenta

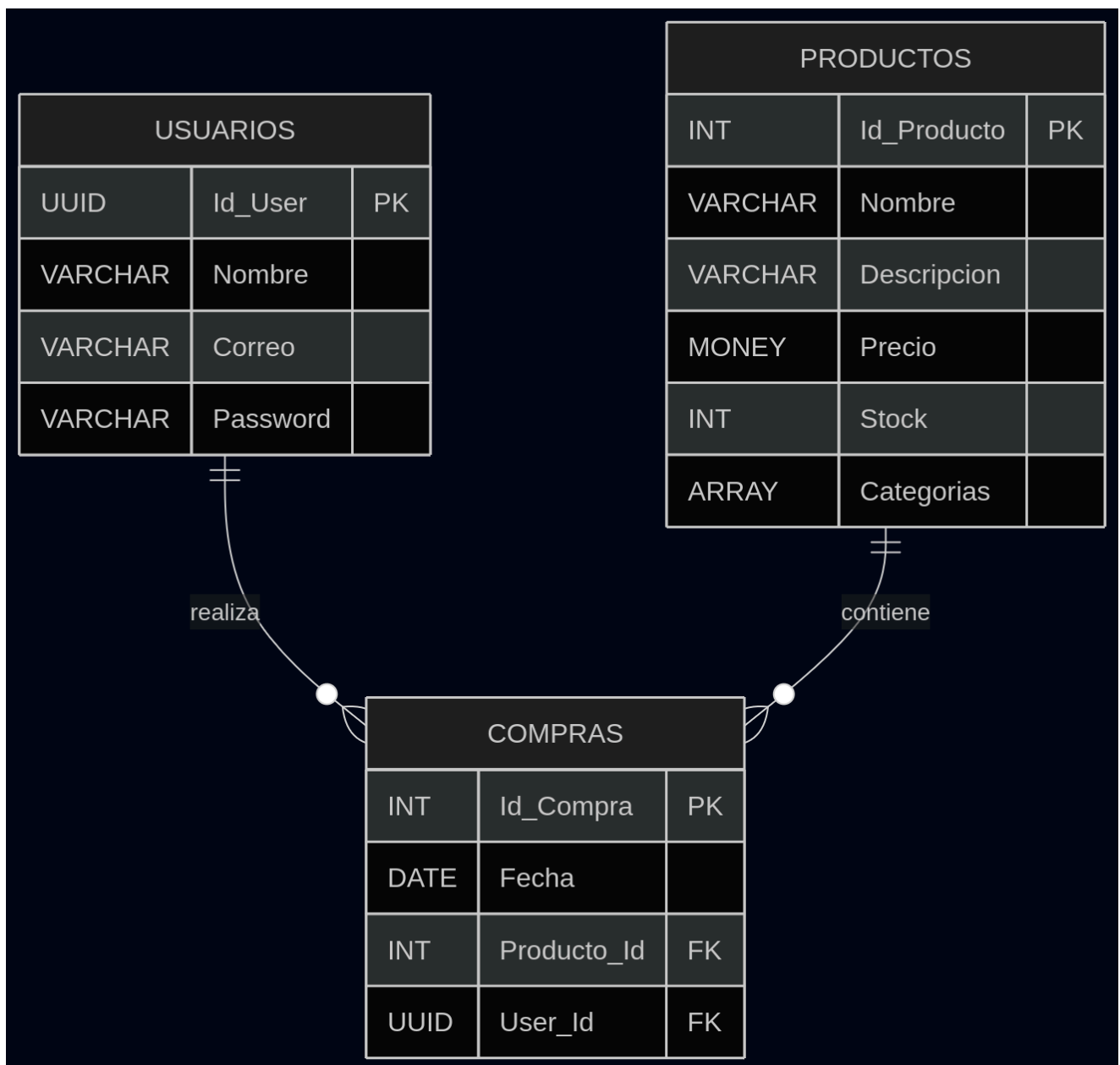
Tabla Productos

Campo	Tipo	Restricciones	Descripción
Id_Producto	INT	PK, Increment.	Identificador interno
Nombre	Varchar	Not null, unique.	Nombre del producto
Descripción	Varchar	Not null	Descripción del producto
Precio	Money	Not null	Precio del producto
Stock	Int	Not null	Cantidad de productos disponibles
Categorías	Array	Not null	Categorías para filtrar el producto

Tabla Compras

Campo	Tipo	Restricciones	Descripción
Id_Compra	INT	PK, Increment	Identificador interno
Fecha	Date	Not null	Cuándo se hizo la compra
Producto_Id	INT	FK, Not null	FK a tabla productos
User_Id	UUID	FK, Not null	FK a tabla usuarios

Entidad-Relación



Diseño de la API:

Importante, cuando se obtienen muchos elementos es necesario poner un límite.

Usuarios:

- `Get /user` Obtiene todos los usuarios.
- `Get /user/:id` Obtiene el usuario por el Id.
- `Post /user` Crea un usuario.
- `Patch /user/:id` Actualiza un usuario.
- `Delete /user/:id` Elimina un usuario.

Productos:

- `Get /producto` Obtiene todos los productos.
- `Get /producto/:id` Obtiene el producto por el Id.
- `Post /producto` Crea un producto.
- `Patch /producto/:id` Actualiza un producto. (Ejemplo: solo el precio)
- `Delete /producto/:id` Elimina un producto.

Compras:

- `Get /compra` Obtiene todas las compras.
 - `Get /compra/:id` Obtiene la compra por el Id.
 - `Post /compra` Crea una compra.
 - `Patch /compra/:id` Actualiza una compra.
 - `Delete /compra/:id` Elimina una compra.
-

Estructura de carpetas:

Archivos y carpetas a crear

-  EAcommerce
 -  src
 -  config
 -  controllers
 -  middleware
 -  routes
 -  services
 -  app.js
 -  auth.js
 -  frontend
 -  src
 -  components
 -  pages
 -  services
 -  App.jsx/css
 -  index.css
 -  main.jsx

-  tailwind.config.js
-

Prioridades:

Must have:

- Base de datos robusta que tenga las tres tablas principales. Importante permitir que la contraseña esté encriptada.
- Creación de usuario e inicio de sesión.
- Roles: solo los administradores pueden crear productos.
- Tarjetas para cada producto.
- Filtros y búsqueda para productos.
- Una pestaña con información para cada producto.
- Un sistema de transacciones para simular compras.
- Carrito.

Should:

- Recomendaciones de otros productos en el carrito.
- Feed en el home.
- Compras "reales".
- Recibo de compras.
- Un chatbot o alguna manera de contactar soporte.

Could:

- Otros métodos de inicio de sesión.
- Gestión de la devolución de productos.

Won't:

- Uso de criptomonedas.
 - Envío a domicilio.
 - Productos por parte de los usuarios.
-

Flujo de datos:

- El usuario llega a la parte de login y manda una solicitud para loguearse (o para crear una cuenta).
- El servidor valida su usuario y contraseña (o su intento de creación).
- Si se loguea, el usuario tiene que hacer un fetch de varios productos.

- Si busca palabras clave o pone filtros, tiene que buscar esos otros productos.
- Puede agregar los productos a un carrito.
- Puede continuar con la compra.
- Hacer las validaciones necesarias.
- Compra lista, añadir a tabla compras.

